



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA E
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Vargas Luna José Ángel

N° de Cuenta: 318252333

GRUPO DE LABORATORIO: 07

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 18-04-2026

CALIFICACIÓN: _____

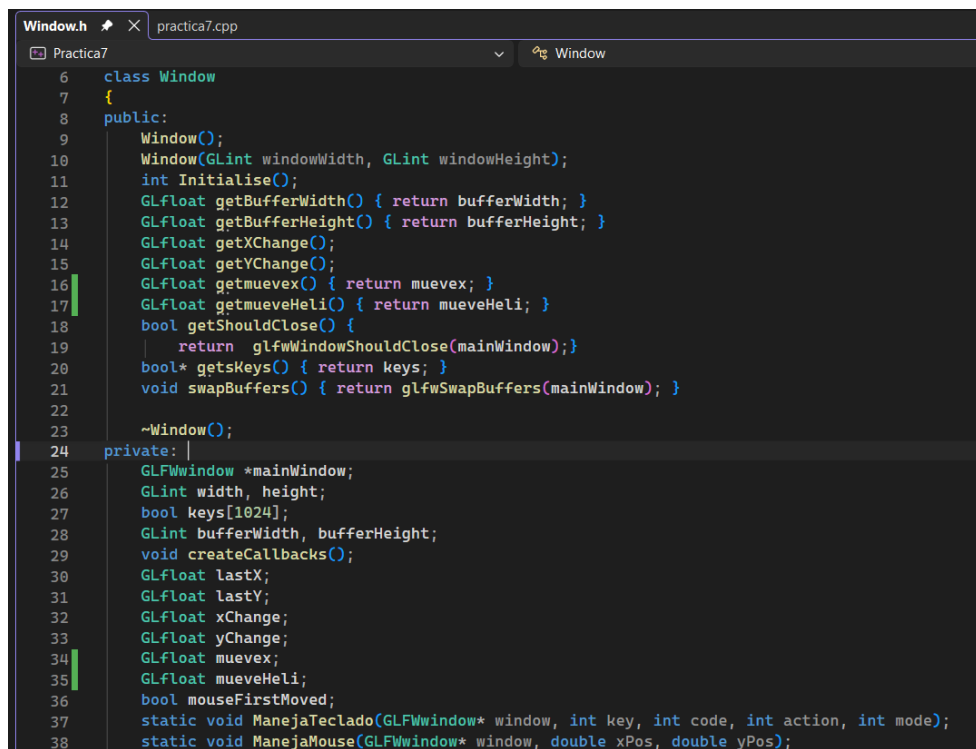
1.-Agregar movimiento con teclado al helicóptero hacia adelante y atrás.

Para este apartado de mi práctica primero crearemos un variable tipo Float para que se pueda crear el movimiento del helicóptero.

```
66 GLfloat deltaTime = 0.0f;
67 GLfloat lastTime = 0.0f;
68 GLfloat MoviHelicoptero = 0.0f;
69 static double limitFPS = 1.0 / 60.0;
```

Con el GLfloat MoviHelicoptero declararemos que nuestro objeto tendrá movimiento.

Después en nuestro archivo Window.h y Window.cpp haremos unas leves modificaciones. Primero lo haremos en Window.h donde agregaremos la variable GLfloat mueveHeli, tanto en **public:** como en **private:**



```
Window.h practica7.cpp
Practica7 Window
6 class Window
7 {
8 public:
9     Window();
10    Window(GLint windowWidth, GLint windowHeight);
11    int Initialise();
12    GLfloat getBufferWidth() { return bufferWidth; }
13    GLfloat getBufferHeight() { return bufferHeight; }
14    GLfloat getXChange();
15    GLfloat getYChange();
16    GLfloat getmuevex() { return muevex; }
17    GLfloat getmueveHeli() { return mueveHeli; }
18    bool getShouldClose() {
19        return glfwWindowShouldClose(mainWindow);}
20    bool* getsKeys() { return keys; }
21    void swapBuffers() { return glfwSwapBuffers(mainWindow); }
22
23    ~Window();
24 private:
25    GLFWwindow *mainWindow;
26    GLint width, height;
27    bool keys[1024];
28    GLint bufferWidth, bufferHeight;
29    void createCallbacks();
30    GLfloat lastX;
31    GLfloat lastY;
32    GLfloat xChange;
33    GLfloat yChange;
34    GLfloat muevex;
35    GLfloat mueveHeli;
36    bool mouseFirstMoved;
37    static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
38    static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
```

Para después en Window.cpp agregaremos las variables de las teclas P & L para que el helicóptero se mueva hacia adelante y atrás, en mi caso, documente el código del carro para poder visualizar mejor al helicóptero.

```
Window.cpp practica7.cpp (Ámbito global)

if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
{
    glfwSetWindowShouldClose(window, GL_TRUE);
}
if (key == GLFW_KEY_Y)
{
    theWindow->muevex += 1.0;
}
if (key == GLFW_KEY_U)
{
    theWindow->muevex -= 1.0;
}
if (key == GLFW_KEY_P)
{
    theWindow->mueveHeli += 1.0; // Adelante
}
if (key == GLFW_KEY_L)
{
    theWindow->mueveHeli -= 1.0; // Atrás
}
```

Aquí declaramos nuestras nuevas teclas para el movimiento del helicóptero.

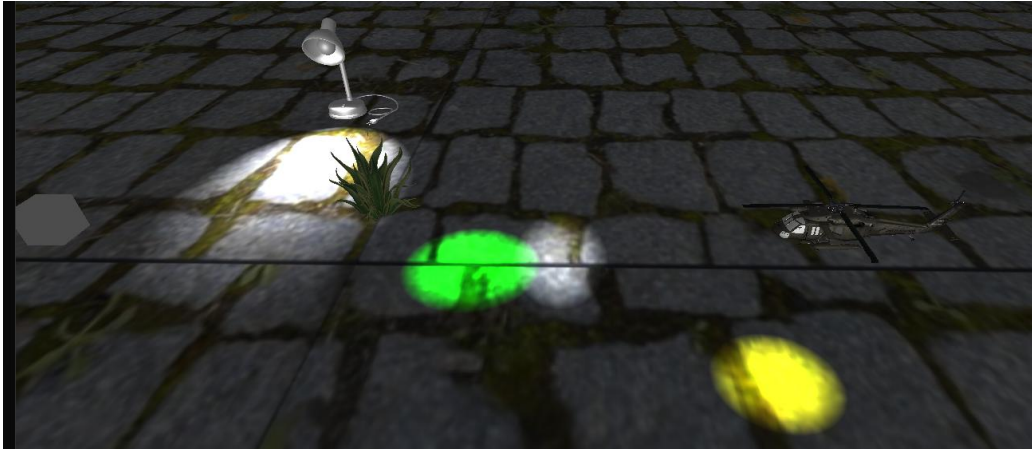
Finalmente en nuestro código principal mandamos a llamar a la función que moverá al helicóptero.

```
64
65 //Sphere cabeza = Sphere(0.5, 20, 20);
66 GLfloat deltaTime = 0.0f;
67 GLfloat lastTime = 0.0f;
68 GLfloat MoviHelicoptero = 0.0f;
69 static double limitFPS = 1.0 / 60.0;
70
```

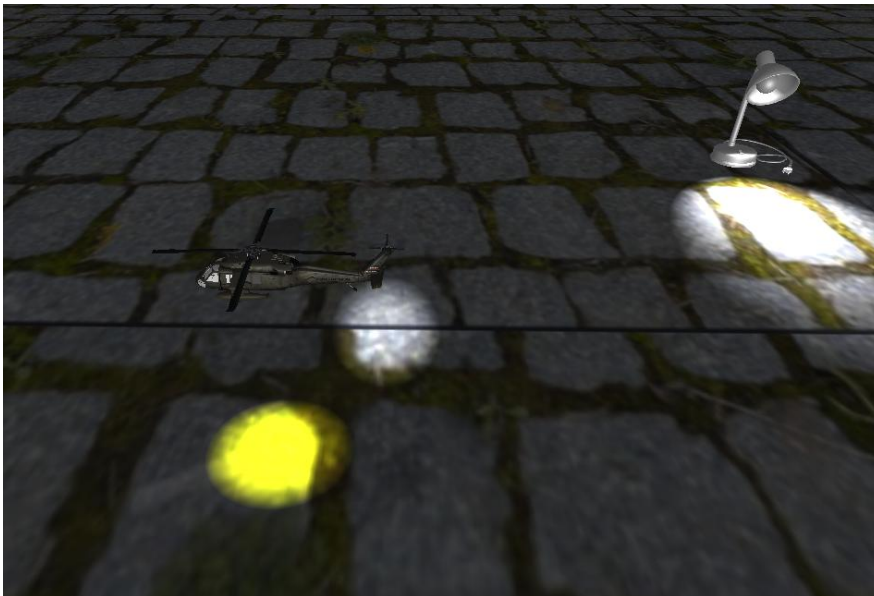
La inicializamos en cero para evitar que agarre una posición random o aleatoria.

```
*/
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmueveHeli(), 5.0f, 6.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
// Estas dos rotaciones son las que orientan al helicóptero
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Blackhawk_M.RenderModel();
```

En esta parte del código, llamamos nuestro modelo y además agregamos una rotación que el modelo del carro no tiene, esto ayuda bastante para que nuestro helicóptero se pueda mover adecuadamente.



Si tomamos como referencia al agave que estaba frente a el, vemos al helicóptero más atrás de el, mostrando que se recorrió hacia atrás.



En esta captura el helicóptero se mueve hacia adelante.

2.-crear luz spotlight de helicóptero de color amarilla que apunte hacia el piso y se mueva con el helicóptero

```
//Luz Del helicóptero
spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f, //Color amarillo de la luz
    0.3f, 2.0f,
    0.0f, 5.0f, 6.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
```

Si nos vamos por las filas de este spot, hacemos lo siguiente:

1. Declaramos el color de la luz.
2. Intensidad
3. Posición, la misma que la del helicóptero, no encima, sino más bien que esté debajo de ello.
4. Dirección, debe dirigirse hacia abajo.
5. Atenuación
6. Ángulo

Con este código dentro del ciclo while haremos que la luz se actualice de posición, todo dependiendo de dónde se encuentre el helicóptero, haciendo que la luz siempre se mueva con respecto al helicóptero

```
glm::vec3 posHeli = glm::vec3(0.0f + mainWindow.getMueveHeli(), 5.0f, 6.0f);  
// 2. Actualizamos la posición de la luz amarilla  
spotLights[2].SetPos(posHeli);
```

Sin embargo, si ejecutamos el programa así, obtendremos el siguiente resultado.



Nuestra luz de ambiente es tan brillante (como la del Sol), que la luz del helicóptero no se ve, entonces lo que debemos hacer es modificar la luz de ambiente a una más tenue o más opaca que nos permita ver la luz del helicóptero. En la carpeta de Shaders, modificaremos el archivo `shader_light.frag` donde comentaremos este código.

```
color = texture(theTexture, TexCoord)*vColor;
```

Y es su lugar pondremos el que se encuentra abajo documentado, dándonos como resultado el siguiente código.

```
//color = texture(theTexture, TexCoord)*vColor;
color = texture(theTexture,
TexCoord)*vColor*finalColor;
```

Esto hará nuestra luz de ambiente más opaca, permitiéndonos ver la luz del helicóptero.



3.- Añadir en el escenario 1 modelo de lámpara texturizada y crearle luz puntual blanca

Primero meteremos nuestro modelo en la carpeta Models y la mandaremos a llamar.

```
50
51     Model Kitt_M;
52     Model Llanta_M;
53     Model Blackhawk_M;
54     Model Lampara;
55
```

```
Lampara = Model();
Lampara.LoadModel("Models/Lampara.obj");
```

Ahora crearemos una spotlight para la lámpara, algo que debemos tomar a consideración es que el foco de la lámpara tiene ángulo de inclinación, por lo que al codificarla deberemos hacer que la luz tenga ángulo y no un punto fijo hacia el suelo, como lo que ocurre con el helicóptero.


```

pointLightCount = 0;
//Luz de la lámpara
spotLights[3] = SpotLight(1.0f, 1.0f, 1.0f, // Color Blanco
    0.0f, 4.0f,
    posLampara.x, posLampara.y + 4.5f, posLampara.z, // Posición del foco
    0.0f, -1.0f, 0.8f, //Inclinación de la luz, efecto que hace que la lámpara se vea más real
    1.0f, 0.0f, 0.0f,
    35.0f);
spotLightCount++;

```

Después una vez declarado su estado de la luz, deberemos llamar a nuestra lámpara y ajustarle sus datos, algo importante es que como mi modelo ya se encuentra derecho, no es necesario agregarle el rotate.

```

//Lámpara importada
model = glm::mat4(1.0);
model = glm::translate(model, posLampara);
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
Lampara.RenderModel();

```

Finalmente cuando lo ejecutemos, como el shader_light.frag lo modificamos se verá el efecto de la luz en mi modelo.



Conclusiones

La elaboración de esta práctica fue muy útil, ya que reutilizamos todo lo visto en clase pasadas, tales como la implementación de modelos, manejo de los objetos con el teclado, texturizado y ahora agregamos el uso de iluminación, haciendo más divertido y realista nuestros objetos y paisajes, como el caso del helicóptero hacerlo más realista el modelo con la luz.

Referencias

G-Truc Creation. (s.f.). *OpenGL Mathematics (GLM) Documentation*. <https://glm.g-truc.net/>

Phong, B. T. (1975). Illumination for computer generated pictures. *Communications of the ACM*, 18(6), 311-317. <https://doi.org/10.1145/360825.360839>

Khronos Group. (2026). *The OpenGL Shading Language (GLSL) Specification, Version 4.60*. <https://www.khronos.org/registry/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>